
A Sprint de um QA: qualidade de software na prática. Dia 2 · Execução, API, automação e CI/CD.

Lucas da Cunha Rodrigues · UDESC · 07/10.

Onde paramos



Step 1

Você escolheu uma história do **Inscrevi**



Step 2

Refinou com o **INVEST**



Step 3

Escreveu os cenários em **Dado, Quando, Então**



Step 4

Montou o **plano de teste**

Hoje esse material vira **execução**

>> █

0 fio de hoje



IA como copiloto



1

Um prompt pronto para cada etapa, em `dia-2/prompts`



2

A IA sugere, você decide



3

Não cole dado sigiloso



4

Não peça para o teste passar

Executar um caso de teste

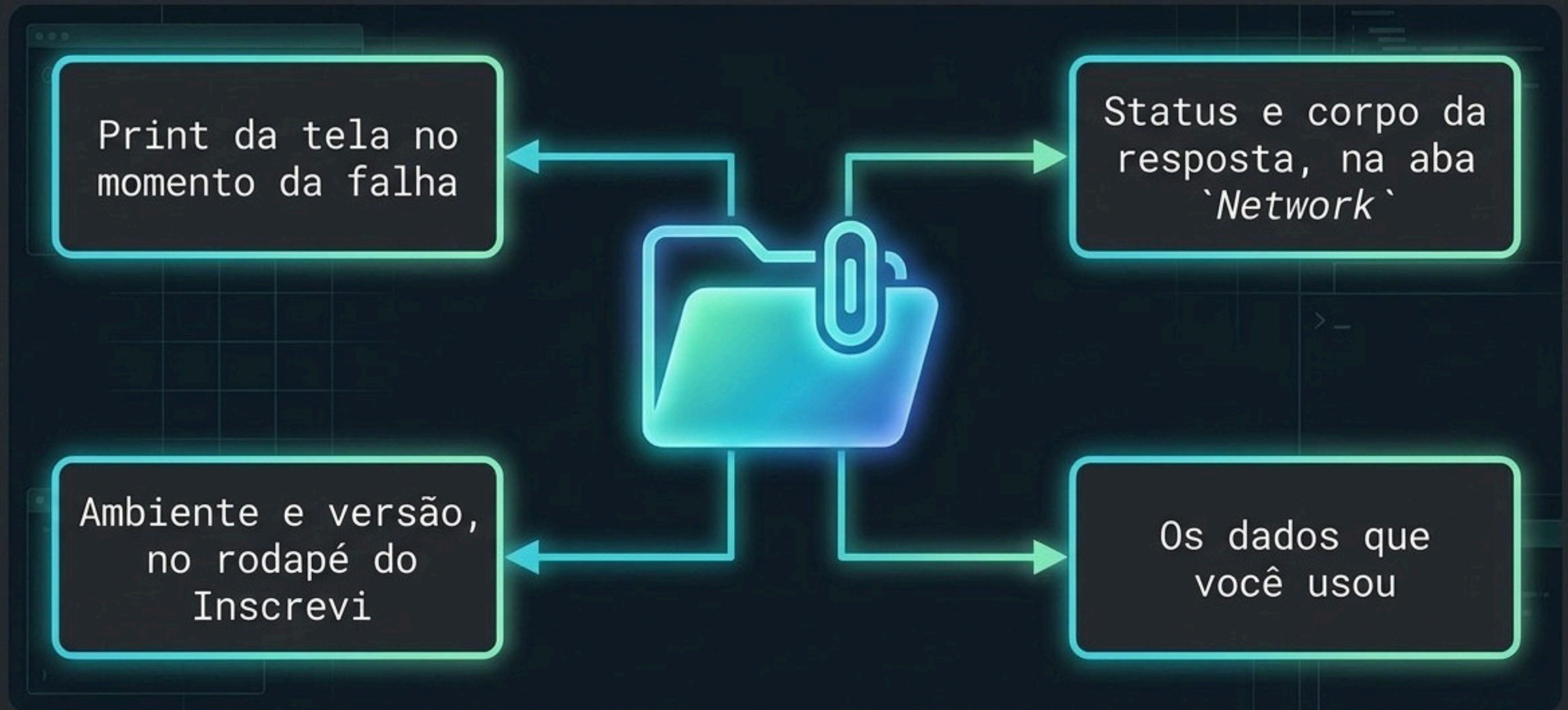
Dado: monte a
pré-condição

Quando:
execute a
ação com os
dados do caso

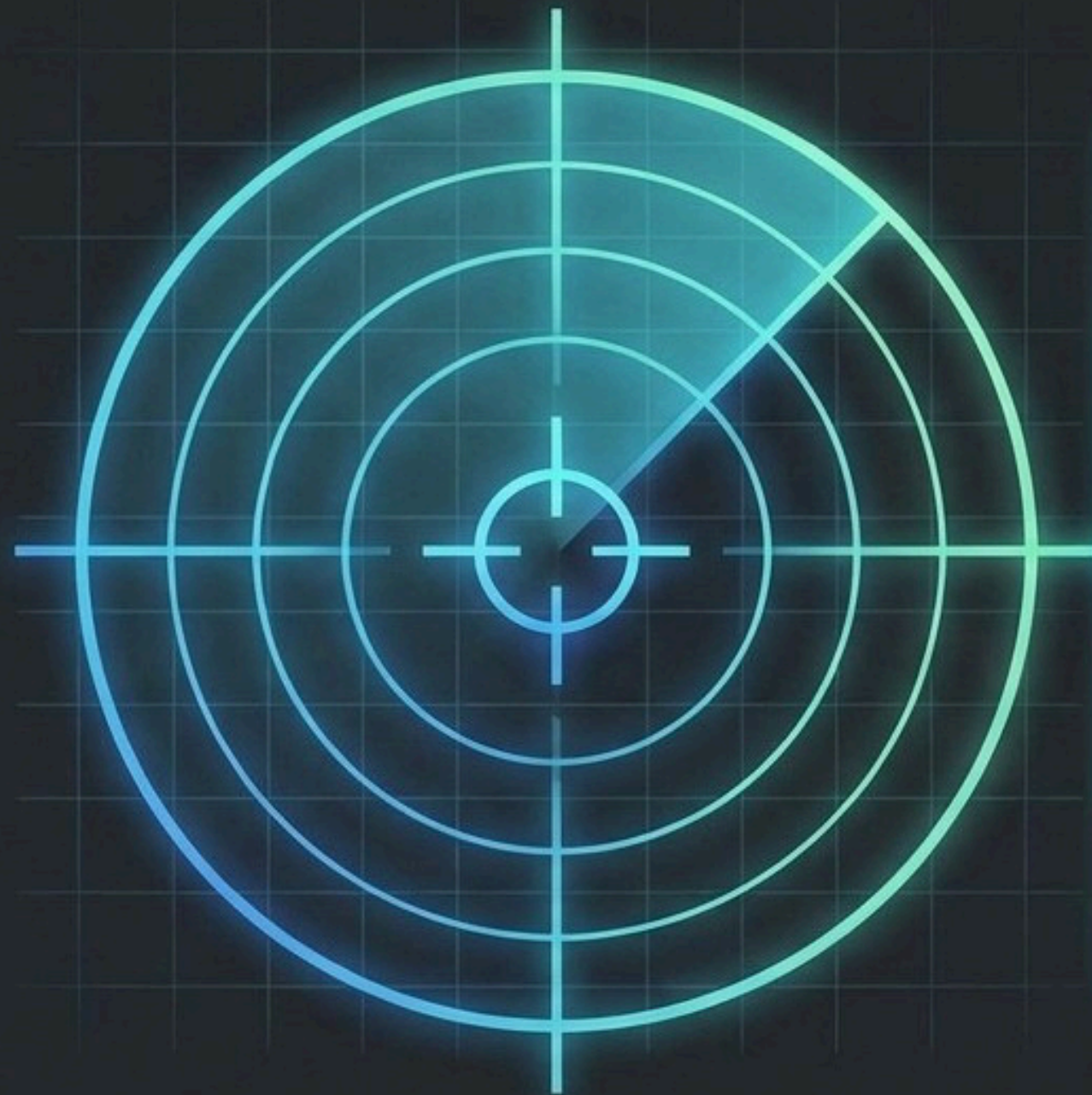
Então:
compare o
obtido com o
esperado

Registre:
passou,
falhou ou
bloqueado

Evidência na hora



Explorar com um charter



Uma missão, um tempo fixo, anotações

Valores nas bordas e logo depois delas

A mesma ação duas vezes seguidas

Outra conta, ou sem estar logado

O botão Voltar e o endereço digitado à mão

Anatomia de um bom relato de bug

- 1 **Título:** [área] - [o que aconteceu] - [em que condição]
- 2 **Passos numerados**, com os dados usados
- 3 **Esperado**, citando o critério de aceite
- 4 **Obtido**, como fato, sem hipótese de causa
- 5 **Evidência**, ambiente e versão

Severidade não é prioridade

Severidade

quanto o defeito afeta
o sistema (técnica)

Prioridade

quão rápido precisa ser
corrigido (negócio)



Erro de digitação na página inicial, na véspera do evento: severidade baixa, prioridade alta

IA na execução e no bug

Prompt 03: preparar a execução e as ideias de exploração



Confira: a IA manteve o fato ou “suavizou”?

Prompt 04: escrever ou revisar o relato



Confira: os passos são os que você executou?

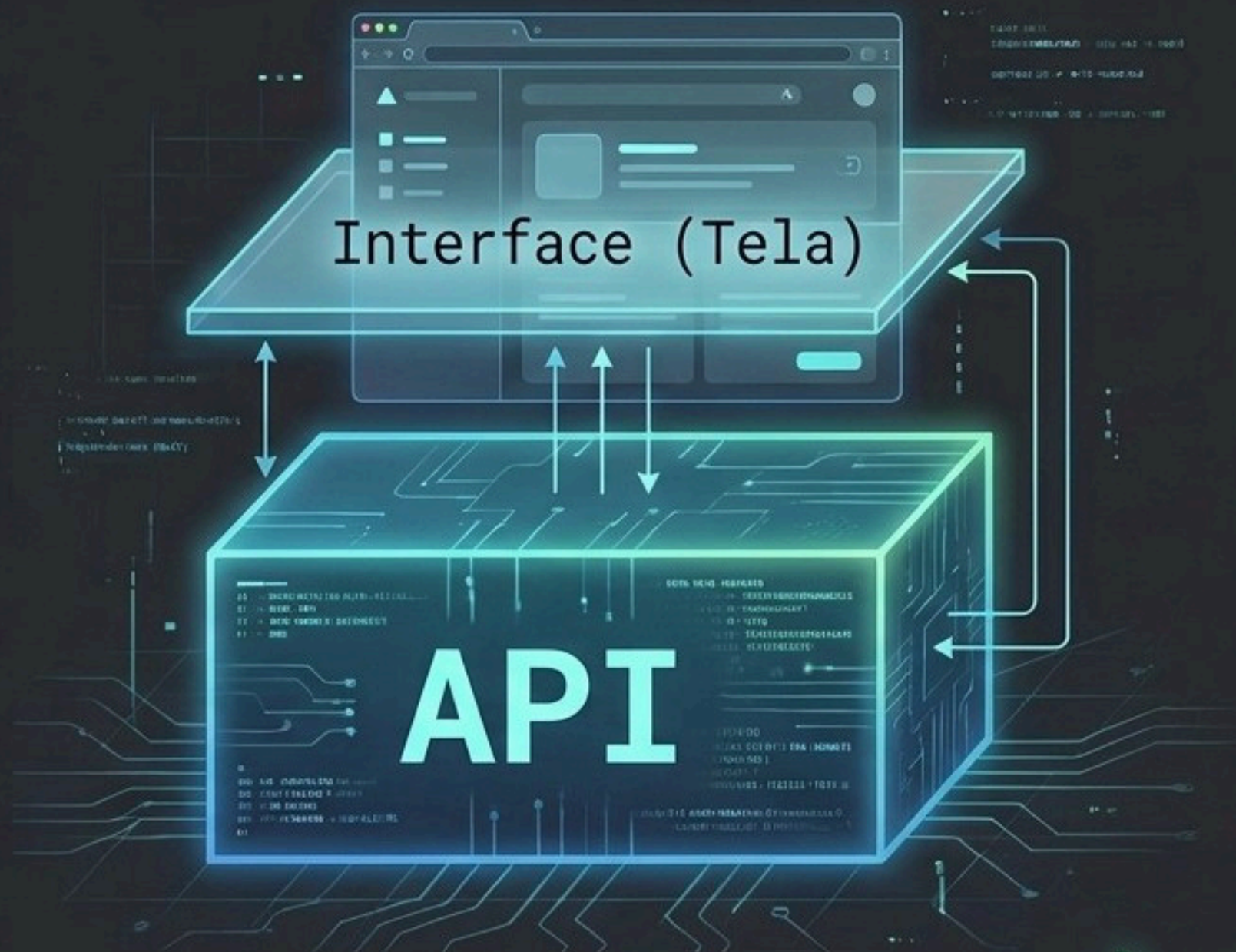
Por que testar a API

A tela é só uma das portas de entrada

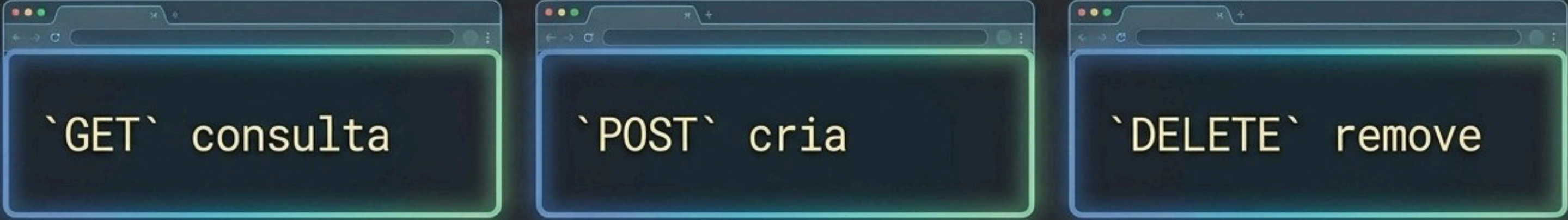
Uma regra que a tela protege pode estar aberta na API

Mais rápido e mais estável que testar pela tela

Encontra o defeito mais perto da causa



HTTP em um slide



Three browser window icons are shown, each with a different colored border (blue, green, and red). Each window contains text describing an HTTP method and its function.

``GET` consulta`

``POST` cria`

``DELETE` remove`

- ``2xx`` deu certo: ``200``, ``201`` criou, ``204`` sem corpo

- ``4xx`` erro de quem chamou: ``400``, ``401``, ``403``, ``404``, ``409``

- ``5xx`` erro do servidor

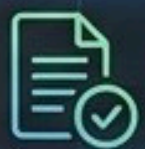
O que conferir numa resposta

● 200
OK

Status HTTP

{ }

Corpo: campos, tipos e mensagem de erro



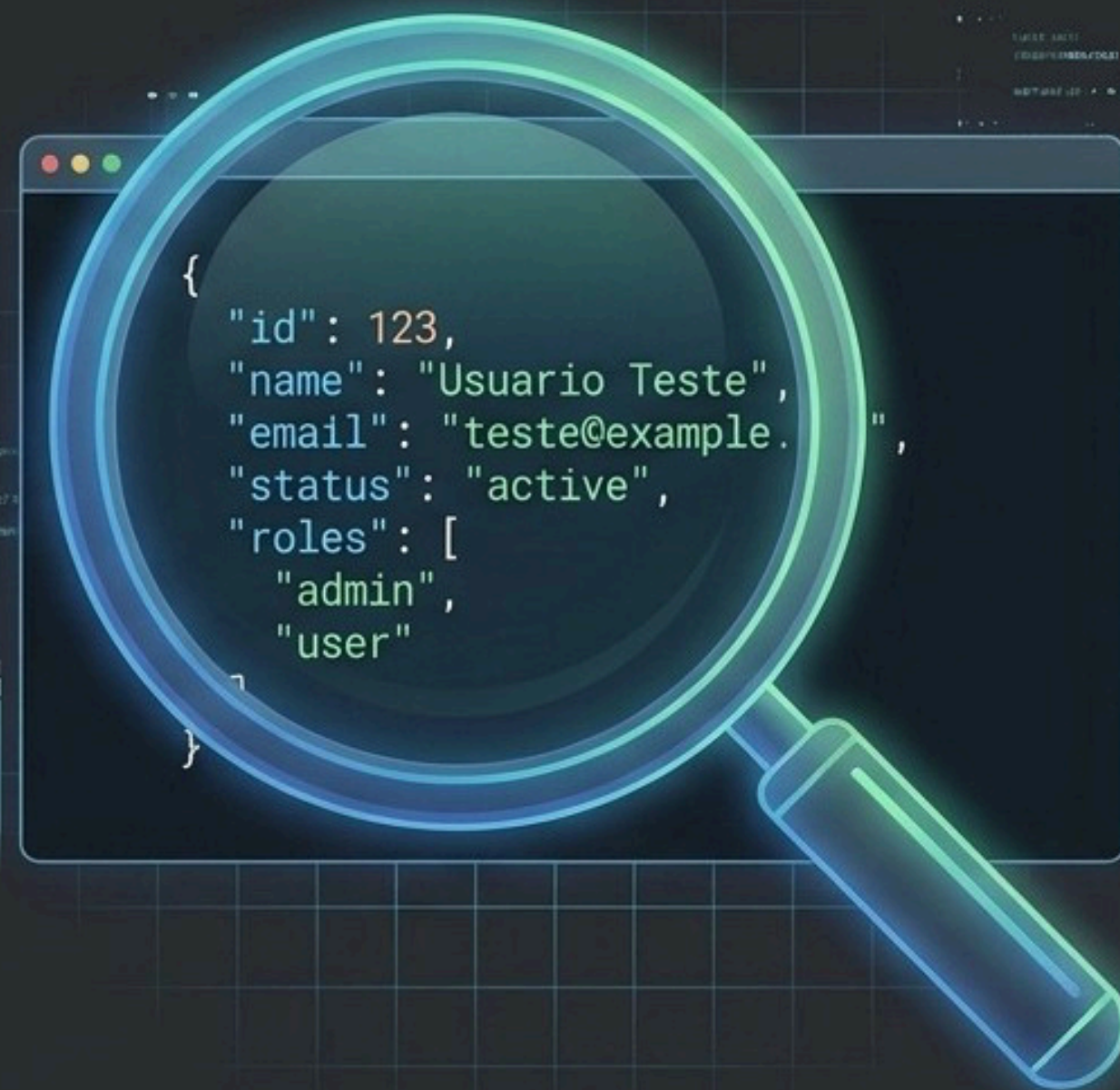
Contrato: bate com a documentação?



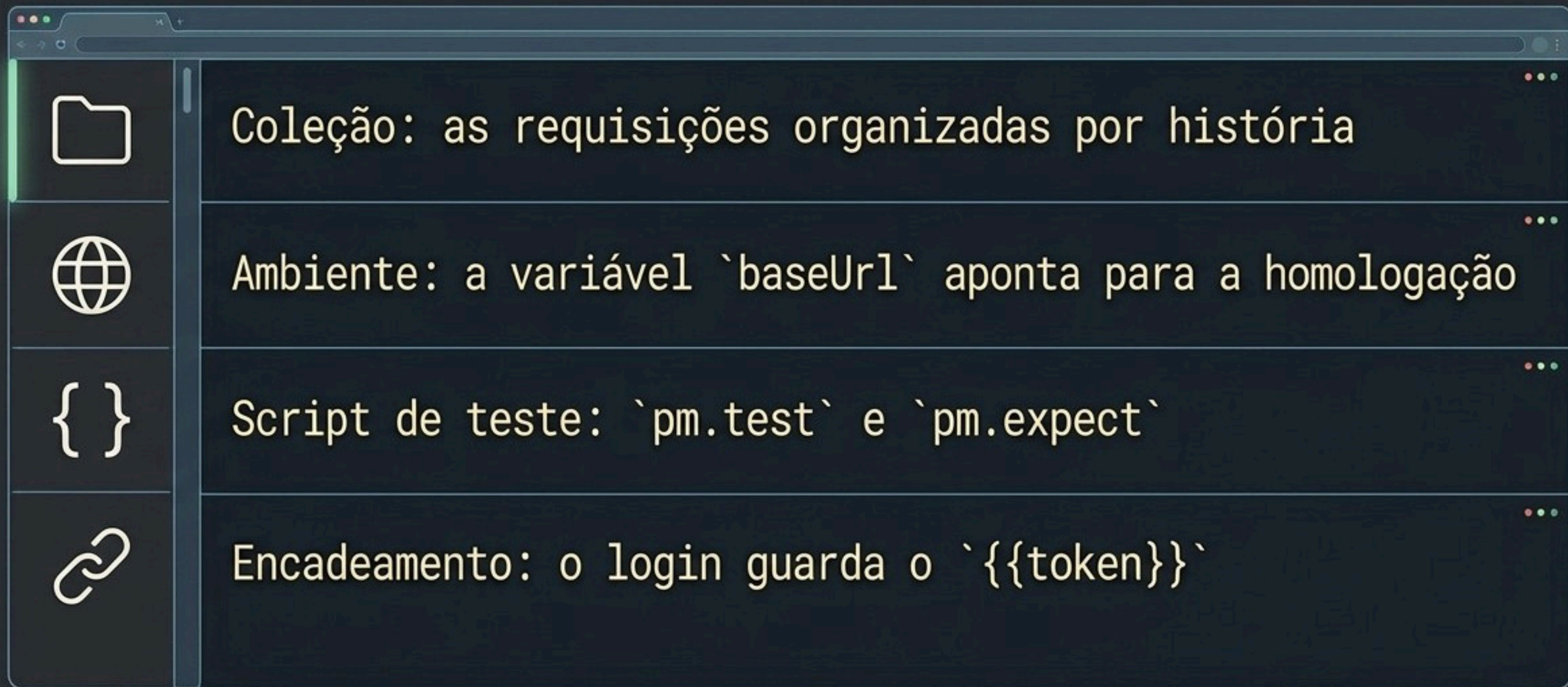
Efeito: o dado mudou mesmo?



Autenticação e autorização



Postman: coleção, ambiente e scripts



Falha de ambiente ou defeito?



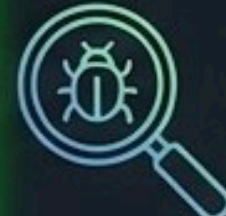
Reiniciar os dados responde 403: só o instrutor pode



O tempo de resposta varia com a rede



A contagem de vagas pode não bater: o ambiente é compartilhado



Antes de abrir um bug: a pré-condição estava certa?

IA nos testes de API



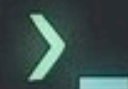
Prompt 05: cenários e scripts para a sua rota

Cole a história e o trecho da documentação



Confira:

- ☒ Confira: o status esperado vem do critério de aceite?
- ☒ Confira: a mensagem de erro existe mesmo?



Por que automatizar

O "PORQUÊ"

- ✓ Para repetir a cada entrega sem custo
- ✓ Para ter resposta em minutos
- ✓ Automatize o que é estável, repetitivo e importante

LIMITES

- ✗ Não automatize o que ainda está mudando
- ✗ Exploração continua sendo humana



Cypress em um slide

Ferramenta de teste que roda dentro do navegador

``npx cypress open``: com interface, para escrever e depurar

``npx cypress run``: headless, para conferir e para o pipeline



Espera sozinho: nada de pausa com tempo fixo